GoldenSetAuditor

Product Requirements Document — v0.2.0

Evaluation dataset quality auditor for LLM / RAG applications

Field	Value
Status	Released — v0.2.0
Author	Sidharth Kriplani
Language	Python 3.10 / 3.11 / 3.12
License	MIT
Dependencies	pandas, numpy
Tests	13 unit tests, 4 test classes
PyPI	goldensetauditor

---

1. Problem Statement

Golden sets — curated question/answer pairs used to benchmark LLM and RAG applications — are the foundation of offline evaluation. A golden set that contains structural quality problems will produce benchmark scores that are misleading regardless of model quality.

Common failure modes in golden sets:

• **Conflicting labels** — the same prompt has different expected answers, making ground truth undefined

• **Duplicate examples** — repeated prompts inflate coverage metrics and bias fine-tuned models

• **Weak expected answers** — short or near-empty reference answers that cannot discriminate between good and bad model outputs

• **Ambiguous prompts** — questions with anaphoric references or multiple sub-questions that lack a single correct answer

• **Over-easy examples** — prompts that contain the answer verbatim, rewarding retrieval over reasoning

• **Near-duplicates** — paraphrased prompts that reduce diversity without adding evaluation signal

• **Category imbalance** — underpopulated categories that produce unreliable per-group accuracy estimates

These problems are common because golden sets are often assembled by automated pipelines or manual annotation under time pressure, without a systematic quality gate. They are typically discovered only after scores seem implausible or after a post-mortem on a failed model release.

2. Goals and Non-Goals

Goals:

• Provide a single function call that audits a pandas DataFrame for seven known golden-set quality failure modes

• Return structured, machine-parseable findings (PASS/WARN/FAIL/INSUFFICIENT_INPUT) before benchmark scores are trusted

• Make the evaluation dataset review step systematic, documentable, and repeatable

• Distinguish structural violations (FAIL: label conflict) from statistical suspicions (WARN) with appropriate severity

• Produce three output formats: JSON for CI gates, Markdown for documentation, HTML for human review

Non-Goals:

• Evaluating model answers or output quality — this tool audits the dataset, not the model

• Measuring inter-annotator agreement or annotation calibration

• Verifying factual correctness or currency of expected answers

• Multi-turn conversation auditing (scoped as future capability)

• Making automatic go/no-go decisions — human review of findings is required

3. Target Users

User	Use case
LLM/RAG Engineer	Run audit before publishing benchmark scores; catch dataset issues before they become explana
Evaluation Lead / ML Researcher	Review audit report as part of golden-set sign-off process
ML Platform / MLOps team	Integrate audit as CI gate in evaluation pipeline; fail builds on FAIL status
Interview candidate	Demonstrate systematic evaluation engineering practice; show structured audit artifact

4. Check Specifications

Check	Config fields	Default	FAIL condition	WARN condition
Conflicting duplicate	input_col, expected_col	—	Same prompt, different expected answers	
Exact duplicate	input_col, expected_col	—	—	Same prompt, same expected answer
Weak expected answer	expected_col, min_expected_tokens	4 tokens	—	Meaningful token count < threshold
Ambiguous prompt	input_col	—	—	Anaphoric ref in short prompt OR multipl
Over-easy example	input_col, expected_col, overeasy_overlap_threshold	0.72		Token Jaccard(prompt, answer) >= thres
Near-duplicate scan	input_col, near_duplicate_threshold, max_pairwise_checks	0.82 / 25,000		Pairwise Jaccard >= threshold; INSUFFI
Category coverage	category_col, min_category_count	8	—	Category count < threshold; INSUFFICIE

5. Input / Output Model

Input:

• **df**: pandas DataFrame — the golden evaluation dataset to audit

• **config**: GoldenSetAuditConfig dataclass:

Field	Type	Required	Description
input_col	str	Yes	Column containing prompt / question text (default 'question')
expected_col	str	Yes	Column containing reference answer (default 'expected_answer')
category_col	str | None	No	Column with category labels (enables category coverage check)
id_col	str | None	No	Column with row identifiers for traceable findings
min_expected_words	int	No	Minimum meaningful tokens in expected answer (default 4)
near_duplicate_threshold	float	No	Token Jaccard threshold for near-duplicate scan (default 0.82)
overeasy_overlap_threshold	float	No	Token Jaccard threshold for over-easy detection (default 0.72)
min_category_count	int	No	Minimum examples per category (default 3)
max_pairwise_checks	int	No	Max pairs for near-duplicate scan (default 25,000)

Output — GoldenSetReport:

Field	Type	Description
status	str	PASS / WARN / FAIL / INSUFFICIENT_INPUT — worst finding across all chec
findings	list[GoldenSetFinding]	One entry per detected issue (check_name, status, severity, row_ids, detail, rec
summary	dict	rows, columns, finding_count, warn_count, fail_count, insufficient_input_count
to_json()	str	Full report as JSON string
to_markdown()	str	Human-readable Markdown report with interpretation note
to_html()	str	Self-contained HTML report with truth boundary note
save(path, stem)	None	Writes .json, .md, .html to path with given filename stem

6. Public API

Minimal usage:

```
from goldensetauditor import GoldenSetAuditConfig, audit_golden_set
report = audit_golden_set(df, GoldenSetAuditConfig())
print(report.status) # PASS / WARN / FAIL
report.save('outputs/') # writes JSON, Markdown, HTML
```

Full config usage:

```
config = GoldenSetAuditConfig(
    input_col='question',
    expected_col='expected_answer',
    category_col='category',
    id_col='id',
    min_expected_words=5,
    near_duplicate_threshold=0.80,
    min_category_count=5,
)
```

7. Test Plan

Test class	Tests	What is covered
GoldenSetAuditorTests	6	Missing columns FAIL, conflicting duplicate FAIL, weak answer WARN, ambiguous promp
NearDuplicateTests	2	Near-duplicate warns at low threshold; distinct questions produce no near-dup finding
CategoryCoverageTests	2	Underpopulated category warns; missing category_col gives INSUFFICIENT_INPUT
EvidenceTests	3	Conflicting duplicate has duplicate_count evidence; report summary has expected keys; sa

All 13 tests pass on Python 3.10, 3.11, 3.12. CI matrix runs on every push via GitHub Actions.

8. Success Criteria

Criterion	Target	Status
All 7 checks implemented	7/7	Done
Unit tests passing	13/13	Done
CI matrix (3.10/3.11/3.12)	All green	Done
Three output formats	JSON + MD + HTML	Done
Zero test dependencies beyond pandas + numpy	Yes	Done
Configurable thresholds via GoldenSetAuditConfig	All 5 thresholds exposed	Done
Explicit truth boundary in every report	Yes	Done
Row-level traceable findings via id_col	Yes	Done
PyPI publish workflow (OIDC trusted publisher)	Configured	Done

Resume-safe claim: Built **GoldenSetAuditor**, an evaluation dataset quality auditor for LLM/RAG applications that checks golden sets for conflicting expected answers, exact and near-duplicate prompts, weak reference answers, ambiguous questions, over-easy examples, and category coverage gaps, producing structured JSON/Markdown/HTML audit reports with per-finding PASS/WARN/FAIL/INSUFFICIENT_INPUT status and explicit truth boundary.